Milestone Report: Safety Verification of Chiron IR using Constrained Horn Clauses

Aditi Khandelia Arush Upadhyaya Kushagra Srivastava

Project Overview

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a PDR-based safety verification engine. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Team Information

Roll No.	**Roll No.**	**Email ID**
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in

Source Code

The complete implementation is hosted at: `https://github.com/chiron-verification/milestone-2`
Commit identifier: `b25038d27bd61a446769546e0aa35dd31a737cba`

All verification code resides under `Chiron-Framework/ChironCore/CHC_Verification/`. The pipeline comprises five modules:

- `variable_name_detection_in_IR.py` — scans the IR to build a symbol table of user variables and a counter table of loop-repetition counters.
- `z3_fixed_point.py` — constructs the `Inv` predicate signature and state-variable tuples for the selected mode.
- `init_fixed_point.py` — instantiates the Z3 `Fixedpoint` object (SPACER) and asserts the base-case fact or universally quantified initial rule.
- `step_rules.py` — translates each IR instruction into a universally quantified CHC implication rule.
- `safety_properties.py` — entry point: builds the fixedpoint system, accepts user-specified properties, and reports PASSED/FAILED/UNKNOWN with invariant or counterexample.

Test programs and pytest suites reside under `CHC_Verification/test_files/`, organised by feature category (`variable_arithmetic/`, `heading/`, `pen/`, `compound/`, `geometric/`). The top-level `test_runner.py` covers 13 programs in `default` mode; `test_files/test_specific_mode.py` provides four tests for `specific` mode.

Current Progress

The implemented pipeline covers the full end-to-end path from a Chiron source program to a property verdict, encompassing state extraction, invariant generation, rule encoding, and property checking.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Literature Review

- **CHCs as a verification IR.** CHC-based encodings yield modular, solver-agnostic verification pipelines [2, 7]; Gurfinkel's notes [3] provide the linear CHC treatment directly applicable here.
- **Z3 Fixedpoint and SPACER.** The μZ engine [5] provides CHC fixed-point computation within Z3; SPACER [1] performs invariant synthesis and reachability queries, returning an inductive invariant or a counterexample.
- **Axiomatic semantics and SeaHorn.** Hoare-style semantics [6] underpin the state-transition encoding; SeaHorn [4] demonstrates the same CHC architecture for C, which our pipeline adapts to Chiron's turtle-graphics semantics.

Implementation Details

Extraction of symbolic program state

File: `CHC_Verification/variable_name_detection_in_IR.py`

The verifier scans the linear Chiron IR to extract all user-declared variables and loop counters introduced by `repeat`, storing each with its corresponding Z3 `Real` variable. Internal repetition counters are kept separate from user variables to avoid name clashes; this determines the arity of the invariant predicate used throughout the pipeline.

Construction of the invariant relation

File: `CHC_Verification/z3_fixed_point.py`

The mode-aware invariant predicate `Inv` captures the full program state: program counter, turtle position `(xcor, ycor)`, heading, pen state, and all user variables and loop counters. Four verification modes are supported:

1. **default**: concrete start state, all values zero, `pendown = False`;
2. **universal**: turtle coordinates, heading, and user variables universally unconstrained;
3. **specific**: user-supplied initial values, with unspecified variables defaulted to zero; and
4. **safety-range**: ghost parameters recording initial user-variable values are appended to `Inv`, enabling the solver's invariant to be read as a precondition over starting values.

Initialization of the CHC system

File: `CHC_Verification/init_fixed_point.py`

The Z3 `Fixedpoint` object (engine: SPACER) is instantiated and seeded with a base-case constraint appropriate to the selected mode: a concrete `fp.fact` for `default`/`specific`, or a universally quantified `fp.rule` for `universal`/`safety-range`.

Translation from Chiron IR to CHC transition rules

File: `CHC_Verification/step_rules.py`

Each IR instruction is mapped to a universally quantified CHC implication over `Inv`:

- **Instruction coverage:** assignments, conditional branching, assertions, `forward`/`backward` movement, `left`/`right` turns, `pendown`/`penup`, `goto`, `pause`, and `NOP`.
- **Expression translation:** arithmetic and boolean expressions are recursively translated to Z3, covering the four arithmetic operations, unary minus, all logical and comparison operators, boolean constants, and pen-status queries.
- **Turtle geometry:** `goto` yields exact coordinate updates; `forward`/`backward` use interval over-approximations of sine and cosine over discretized heading buckets; heading is normalized to $[0, 360)$ via a bounded sequence of conditional rewrites.

Property-checking interface

File: `CHC_Verification/safety_properties.py`

Given a user-provided boolean expression over the state variables, the verifier queries SPACER for a reachable violating state and reports `PASSED`, `FAILED`, or `UNKNOWN`. A counterexample is retained on failure; the

synthesised invariant is stored on success. In `safety-range` mode, the invariant is additionally instantiated at the initial state to extract a symbolic safety precondition over starting values.

Usage

The verifier is invoked from `Chiron-Framework/ChironCore/`. The four supported modes are:

```
uv run python CHC_Verification/safety_properties.py \
    <program.tl> <num_properties> default

uv run python CHC_Verification/safety_properties.py \
    <program.tl> <num_properties> universal

uv run python CHC_Verification/safety_properties.py \
    <program.tl> <num_properties> specific '{"x": 10, "y": 20}'

uv run python CHC_Verification/safety_properties.py \
    <program.tl> <num_properties> safety-range
```

The tool then prompts interactively for property names and boolean expressions over `xcor`, `ycor`, `heading`, `pendown`, user variables, and loop counters; `And`, `Or`, and `Not` are also available.

Description of test files to be used for evaluating the respective milestone

The main evaluation artifacts for this milestone are the automated test modules, the shared test infrastructure, and the `.tl` fixture programs.

Automated test modules

File	Purpose
`tests/test_default.py`	Checks properties in `default` mode, where the entire state starts from a concrete zero initialization. These tests focus on whether the verifier can prove or refute invariants from a fixed start state.
`tests/test_universal.py`	Checks properties in `universal` mode, where user variables and key turtle components are unconstrained initially. These tests validate reasoning under arbitrary initial conditions.
`tests/test_specific.py`	Checks properties in `specific` mode using explicit parameter dictionaries. These tests validate the part of the pipeline that reads concrete input assignments from the command line and injects them into the start state.
`tests/test_safety_range.py`	Checks properties in `safety-range` mode. These tests validate the ghost-parameter mechanism and check whether the implementation can recover a symbolic safe region over initial inputs.
`tests/helpers.py`	Shared testing infrastructure. It builds the fixed-point system for a given program, constructs the property-evaluation context, suppresses verbose pipeline output during testing, and provides helper assertions for expected pass/fail outcomes.
`tests/run_all.py`	Top-level test runner that discovers all `test_*.py` files and executes the complete suite.

Fixture programs used by the tests

Program file	Purpose in evaluation
`assign_basic.tl`	Basic sequential assignments; used to test simple arithmetic invariants and no-motion/no-pen behavior.
`assign_algebra.tl`	Multi-step arithmetic including multiplication and division; used to test algebraic propagation and nonlinear cases.

`conditional.tl`	Simple conditional branch; used to test branching semantics.
`loop_basic.tl`	Counted loop with accumulation; used to test loop invariants.
`loop_accum.tl`	Repetition-driven accumulator growth; used to test monotonic arithmetic invariants.
`loop_cond.tl`	Loop with internal branching and arithmetic updates; used to test interaction between repetition and conditionals.
`goto_computed.tl`	Computed `goto` destinations from symbolic variables; used for geometric safety properties.
`square_goto.tl`	Multiple `goto` commands that trace a square; used to test bounded-region properties.
`forward_square.tl`	Motion using `forward` and `right`; used for trig-dependent geometric tests.
`turns_only.tl`	Pure heading updates; used for directional reasoning and normalization behavior.
`pen_only.tl`	Pen-state updates without arithmetic or motion; used to validate `pendown`/`penup` semantics.
`pen_with_var.tl`	Pen commands combined with variable updates; used to test mixed symbolic and boolean state.
`pen_toggle.tl`	Alternating pen state changes; useful for repeated pen-state transitions.
`param_goto.tl`	Parameterized `goto`; used in `specific` and `safety-range` modes.
`param_loop.tl`	Loop whose initial value depends on a parameter; used to test parameter-sensitive arithmetic invariants.
`param_cond.tl`	Branching behavior controlled by a parameter; used to test mode-dependent initialization and branch selection.
`param_scale.tl`	Arithmetic scaling with an input parameter; used to test concrete parameter injection and sign-sensitive properties.
`param_pen.tl`	Parameterized arithmetic followed by pen commands; used to test the interaction of user inputs and pen state.
`read_before_write.tl`	Reads an input parameter before any local assignment; used to validate handling of externally supplied variables in `specific` mode.

How the test files should be used during evaluation

For milestone evaluation, the recommended procedure is:

1. run the automated suite with `python run_all.py`;
2. inspect individual mode-specific modules when needed; and
3. use representative fixture programs directly with `safety_properties.py` to demonstrate interactive verification in `default`, `universal`, `specific`, and `safety-range` modes.

This combination of direct verifier runs and automated regression tests provides a practical and reproducible way to evaluate the milestone.

Automated validation through unit tests

The repository contains a dedicated automated test suite under `Chiron-Framework/ChironCore/CHC_Verification/tests`. From code inspection, the suite currently contains 89 unit tests spanning the four supported modes. The tests are organized around arithmetic, geometric, pen-state, directional, and safety-range behavior, and they collectively check both expected invariants and expected violations. A shared helper layer suppresses pipeline debug output, constructs the fixed-point system, creates the evaluation context for properties, and provides convenience assertions for pass/fail verdicts.

The full automated suite can be run as follows:

```
cd Chiron-Framework/ChironCore/CHC_Verification/tests
python run_all.py
```

Useful narrower invocations are also supported:

```
# verbose test run
python run_all.py -v
```

```
# run one test module
python -m unittest test_default

# run one class
python -m unittest test_specific.TestSpecificArithmetic

# run one individual test
python -m unittest \
  test_default.TestDefaultArithmetic.test_arith_assign_z_nonneg_pass
```

Overall, the work completed so far establishes a usable verification tool rather than just an isolated encoding: there is a state extractor, a mode-sensitive invariant generator, a step-rule translator, a property interface, and an automated regression suite.

Analysis

Strengths of your implementation

- **Modular design.** Variable extraction, invariant construction, initialization, step-rule encoding, and property checking are separated into dedicated components, making the pipeline easy to read, extend, and debug.
- **Multiple verification modes.** The verifier supports four distinct verification questions — concrete default, universal, user-specified, and symbolic safe-range - within a single unified pipeline.
- **Faithful state modeling.** Both program-level variables and turtle-specific state (position, heading, pen) are tracked, allowing the verifier to reason about motion and turtle semantics, not just numeric values.
- **Broad instruction coverage.** The step-rule encoder handles all main Chiron instruction forms: assignments, branching, assertions, movement, turning, pen control, `goto`, and loop counters.
- **Structured test suite.** Tests are organized by mode and property category and include both positive and negative cases, distinguishing provable invariants from reachable violations.
- **Practical CLI workflow.** A user can invoke the verifier on any `.tl` program, select a mode, and interactively enter properties to obtain pass/fail/unknown verdicts with counterexamples or invariants.

Limitations of your implementation

- **Approximate trigonometric reasoning.** Sine and cosine are non-linear, so movement commands are encoded using interval over-approximations over discretized heading buckets. This increases solver burden, reduces precision, and already causes heading-dependent tests to be skipped in the automated suite because the resulting encoding is too complex for `SPACER` to handle efficiently.
- **Heading normalization adds encoding complexity.** Normalizing heading to $[0, 360)$ via a bounded chain of nested `If`-expressions inflates the CHC encoding and makes invariant synthesis harder for the solver, particularly when turns are combined with forward or backward motion.
- **Nonlinear arithmetic causes solver timeouts.** Programs involving multiplication of symbolic quantities generate nonlinear constraints that `SPACER` cannot always resolve, producing `UNKNOWN` instead of a definitive verdict.
- **Test programs are small and pedagogical.** The current benchmark suite covers feature breadth but not depth; no program has been stress-tested with deep nesting, large loop counts, or complex symbolic properties, so scalability remains unvalidated.
- **Properties tested are too simple.** Most verified properties are direct bounds or simple equalities. More realistic safety requirements — combining heading, position, pen state, and user variables across loops and branches — have not yet been formulated or tested.

Data and Visual Analysis

Future Plans

Milestone summary and current status

Milestone 1 Literature review and tooling familiarisation - *completed*: CHC theory studied, Chiron-to-CHC semantics formalised, Z3 Fixedpoint playground implemented.

Milestone 2 End-to-end CHC verification pipeline - *completed*: full IR-to-CHC translation, four verification modes (`default`, `universal`, `specific`, `safety-range`), property-checking interface, CLI, and automated test suite.

Milestone 3 Correctness validation and stress testing - *in progress*: basic regression suite in place; deep-loop and complex-property stress testing, and semantic cross-checks, remain outstanding.

Milestone 4 Final integration, evaluation, and deliverables - *pending*.

Planned work

(a) Pipeline efficiency and trigonometric reasoning

The most pressing technical challenge is the treatment of heading-dependent motion. The current interval-based approximation of sine and cosine over discretized heading buckets is sound but expensive: it inflates the CHC encoding and already causes solver timeouts on heading-heavy programs. The primary goal for the next milestone is to design a more verification-friendly trigonometric encoding. Reducing nonlinear arithmetic in the encoding more broadly (e.g., replacing symbolic multiplications with bounded linear alternatives where possible) is a secondary priority. If time permits, this phase will also include exploration of additional safety property classes.

(b) Canvas-marking safety properties

A distinctive class of safety properties for Chiron concerns the *canvas*: specifically, which regions of the 2D plane will become marked (pen down during motion), and which are guaranteed to remain unmarked. We plan to implement and verify properties of the form: "the turtle never draws in region R", "all drawing is confined to a bounding box", and "the pen is always up when the turtle is in region S". These require reasoning jointly about turtle position, heading, and pen state across all reachable program states, and will constitute a meaningful stress test for the pipeline.

(c) Testing, stress testing, and end-to-end integration

The final milestone centres on robustness and completeness. Planned work includes:

- extending the benchmark suite with programs involving deep nesting, large loop counts, compound arithmetic, and combined geometric and variable-level behavior;
- testing with complex, multi-component safety properties involving both turtle state and user variables;
- reducing the fraction of `UNKNOWN` verdicts through encoding-level improvements and solver parameter tuning; and
- consolidating the full pipeline into a clean, reproducible end-to-end tool with stable CLI outputs, a finalized test suite, and complete documentation.

Declaration

We declare that the work presented in this report is our own and has been carried out in accordance with the guidelines set out by the course. We have not copied any part of this report from any other source, and we have not submitted this report to any other course or institution for assessment. We understand that any violation of this declaration may result in disciplinary action.

References

[1] Generalized pdr with spacer. Online Z3 Guide, 2026. Available at: `https://microsoft.github.io/z3guide/docs/fixedpoints/engineforpdr/`.

[2] Nikolaj Bjørner, Arie Gurfinkel, Kenneth L. McMillan, and Andrey Rybalchenko. Horn clause solvers for program verification. In *Fields of Logic and Computation II*, pages 24–51. Springer, 2015.

[3] Arie Gurfinkel. Constrained horn clauses (chc). ECE 750T Lecture Notes, University of Waterloo, 2018.

[4] Arie Gurfinkel, Temesghen Kahsai, Anvesh Komuravelli, and Jorge A. Navas. The seahorn verification framework. In *Computer Aided Verification (CAV)*, 2015.

[5] Kryštof Hoder, Nikolaj Bjørner, and Leonardo de Moura. μz– an efficient engine for fixed points with constraints. In Ganesh Gopalakrishnan and Shaz Qadeer, editors, *Computer Aided Verification*, pages 457–462, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[6] University of Texas at Austin. Axiomatic semantics and hoare logic. Course Lecture Notes.

[7] Andrey Rybalchenko. Constraint solving for software verification. In *VMCAI*, 2011.